

BAB 3

ENKAPSULASI DAN ACCESS MODIFIER

3.1 Pendahuluan

Pada BAB 2, mahasiswa telah mempelajari konsep dasar pemrograman berorientasi objek, seperti class, object, atribut, method, constructor, dan penggunaan keyword `this`. Pada tahap tersebut, mahasiswa sudah mampu membuat program berbasis objek yang sederhana.

Namun, jika diperhatikan, pada contoh sebelumnya atribut dalam class masih bisa diakses langsung dari luar. Hal ini sebenarnya kurang baik jika program semakin besar, karena data bisa berubah tanpa kontrol dan berpotensi menimbulkan kesalahan.

Pada BAB 3 ini, mahasiswa akan mempelajari bagaimana cara **melindungi data dalam object** agar tidak bisa diakses sembarangan. Konsep ini dikenal dengan istilah **enkapsulasi (encapsulation)**. Selain itu, mahasiswa juga akan belajar tentang **access modifier**, serta penggunaan **getter dan setter** untuk mengatur akses terhadap data.

3.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu memahami dan menerapkan konsep enkapsulasi dalam program Java.

Secara lebih rinci, mahasiswa diharapkan mampu:

1. Menjelaskan pengertian enkapsulasi.
2. Menjelaskan fungsi access modifier dalam Java.
3. Membedakan `public`, `private`, dan `protected`.
4. Menggunakan getter dan setter.
5. Membuat program Java yang menerapkan enkapsulasi.
6. Melakukan validasi sederhana menggunakan setter.

3.3 Pengertian Enkapsulasi

Enkapsulasi adalah konsep dalam OOP yang digunakan untuk **membungkus data (atribut) dan method dalam satu class**, serta mengatur akses terhadap data tersebut.

Dengan enkapsulasi, data tidak bisa diubah secara sembarangan dari luar class. Akses terhadap data dilakukan melalui method tertentu, sehingga lebih aman dan terkontrol.

Sebagai contoh sederhana, bayangkan data nilai mahasiswa. Nilai tersebut tidak seharusnya bisa diubah langsung oleh siapa saja, tetapi harus melalui prosedur tertentu. Dalam program,

hal ini dilakukan dengan cara membuat atribut menjadi `private`, lalu menyediakan method untuk mengaksesnya.

3.4 Mengapa Enkapsulasi Penting

Enkapsulasi memiliki beberapa manfaat penting dalam pemrograman.

Pertama, **melindungi data** agar tidak diubah secara sembarangan. Kedua, membuat program menjadi lebih rapi dan terstruktur karena akses data diatur melalui method. Ketiga, memudahkan pengembangan program karena perubahan pada data dapat dikontrol dari satu tempat.

Dengan menggunakan enkapsulasi, program juga menjadi lebih aman dan mudah dipelihara. Hal ini sangat penting terutama ketika program semakin besar dan kompleks.

3.5 Access Modifier dalam Java

Access modifier adalah kata kunci yang digunakan untuk mengatur **hak akses** terhadap class, atribut, dan method.

Beberapa access modifier yang umum digunakan adalah:

Modifier	Keterangan
public	dapat diakses dari mana saja
private	hanya dapat diakses dalam class yang sama
protected	dapat diakses dalam package yang sama atau subclass
default	tanpa modifier, hanya dalam package yang sama

Pada tahap awal, mahasiswa cukup fokus pada `public` dan `private`.

3.6 Modifier private

Modifier `private` digunakan untuk membuat atribut **tidak bisa diakses langsung dari luar class**.

Contoh tanpa enkapsulasi:

```
class Mahasiswa {  
  
    String nama;  
  
    int nim;  
  
}
```

Sekarang atribut tidak bisa diakses langsung. Untuk mengaksesnya, diperlukan method khusus.

3.7 Getter dan Setter

Getter dan setter adalah method yang digunakan untuk **mengambil dan mengubah nilai atribut**.

- **Getter**

Getter digunakan untuk mengambil nilai atribut.

```
public String getNama() {  
  
    return nama;  
  
}
```

- **Setter**

Setter digunakan untuk mengubah nilai atribut.

```
public void setNama(String nama) {  
  
    this.nama = nama;  
  
}
```

- **Contoh Program Lengkap**

```
class Mahasiswa {
    private String nama;
    private String nim;

    public void setNama(String nama) {
        this.nama = nama;
    }
    public void setNim(String nim) {
        this.nim = nim;
    }
    public String getNama() {
        return nama;
    }
    public String getNim() {
        return nim;
    }
    public void tampilkanData() {
        System.out.println("Nama : " + nama);
        System.out.println("NIM  : " + nim);
    }
}
```

```
public class DemoMahasiswa {
    public static void main(String[] args) {

        Mahasiswa m1 = new Mahasiswa();

        m1.setNama("Andi Saputra");
        m1.setNim("123140001");

        m1.tampilkanData();

    }
}
```

```
public class DemoMahasiswa {
    public static void main(String[] args) {

        Mahasiswa m1 = new Mahasiswa();

        m1.setNama("Andi Saputra");
        m1.setNim("123140001");

        m1.tampilkanData();

    }
}
```

3.8 Validasi dengan Setter

Salah satu kelebihan setter adalah bisa digunakan untuk **validasi data**.

Contoh:

```
class Mahasiswa {  
    private int semester;  
  
    public void setSemester(int semester) {  
        if (semester > 0) {  
            this.semester = semester;  
        } else {  
            System.out.println("Semester tidak valid");  
        }  
    }  
  
    public int getSemester() {  
        return semester;  
    }  
}
```

Dengan cara ini, data yang tidak valid dapat dicegah sejak awal.

3.9 Contoh Studi Kasus

Program sederhana data mahasiswa ITERA:

```
class Mahasiswa {  
  
    private String nama;  
  
    private String nim;  
  
    private int semester;  
  
    public void setName(String nama) {  
  
        this.nama = nama;  
  
    }  
  
    public void setNim(String nim) {  
  
        this.nim = nim;  
  
    }  
  
    public void setSemester(int semester) {  
  
        if (semester > 0) {  
  
            this.semester = semester;  
  
        }  
  
    }  
  
    public void tampilkanData() {  
  
        System.out.println("Nama      : " + nama);  
  
        System.out.println("NIM      : " + nim);  
  
        System.out.println("Semester : " + semester);  
  
    }  
  
}
```

```
public class DemoITERA {  
  
    public static void main(String[] args) {  
  
        Mahasiswa m1 = new Mahasiswa();  
  
        m1.setNama("Dewi Lestari");  
  
        m1.setNim("123140002");  
  
        m1.setSemester(2);  
  
        m1.tampilkanData();  
  
    }  
  
}
```

3.10 Kesalahan Umum

Beberapa kesalahan yang sering terjadi:

- Semua atribut dibuat `public`
- Tidak menggunakan `getter/setter`
- Salah penulisan method `getter/setter`
- Lupa menggunakan `this`
- Setter tidak mengubah nilai atribut

3.11 Tugas Percobaan

Percobaan 1

Buat class `Mahasiswa` dengan atribut `nama`, `nim`, dan `prodi` (private), lalu buat getter dan setter.

Percobaan 2

Buat program untuk menampilkan data mahasiswa menggunakan method.

Percobaan 3

Tambahkan atribut `semester` dan buat validasi agar tidak boleh kurang dari 1.

Percobaan 4

Buat class `MataKuliah` dengan atribut private dan method getter setter.

Percobaan 5

Buat program lengkap data mahasiswa ITERA minimal 3 object.

3.12 Pertanyaan Diskusi

1. Apa itu enkapsulasi?
2. Mengapa atribut sebaiknya dibuat private?
3. Apa fungsi getter dan setter?
4. Apa keuntungan enkapsulasi dalam program?
5. Bagaimana setter bisa digunakan untuk validasi?

3.13 Tugas Laporan Praktikum

Format laporan sama seperti BAB sebelumnya:

1. Cover
2. Tujuan Praktikum
3. Alat dan Bahan
4. Langkah Percobaan
5. Source Code
6. Output Program
7. Analisis
8. Kesimpulan

Laporan dikumpulkan dalam bentuk PDF.

3.14 Penutup

Pada BAB 3 ini mahasiswa telah mempelajari konsep enkapsulasi yang merupakan salah satu dasar penting dalam OOP. Dengan memahami enkapsulasi, mahasiswa dapat membuat program yang lebih aman, rapi, dan terstruktur.

Pemahaman ini akan menjadi bekal penting untuk mempelajari materi selanjutnya seperti **inheritance dan polymorphism** pada bab berikutnya.